

ANÁLISE TÉCNICA APROFUNDADA

Sistema de Nanossatélite CubeSat — PS_OBC

Computador de bordo (OBC), ponte de comunicação e estação
terrena de controlo

ATSAMV71Q21 · Cortex-M7

Firmware bare-metal em C

RP2040 · Bridge/Simulador

Kotlin · Spring Boot

React · TypeScript

OTA · Bootloader dedicado

TMR / SEU

Projeto e Seminário — Licenciatura em Engenharia Informática e de Computadores (ISEL)

Repositórios analisados: **PS_OBC.X** (firmware OBC) · **solder_test** (RP2040) · **dashboard** (estação terrena)

Documento gerado em 21 de julho de 2026

Índice

1. Visão geral do sistema
2. Tecnologias e linguagens utilizadas
3. Arquitetura de software
4. Interfaces de comunicação (I²C, SPI, UART, QSPI)
5. Hardware e respectivas especificações técnicas
6. Fluxo de dados e integração entre componentes
7. Forma de comunicação de cada periférico (protocolos)
8. Vantagens e desvantagens das abordagens (engenharia e prática)

Âmbito. Esta análise cobre os três segmentos do sistema: o **segmento espacial** (firmware do OBC + bootloader OTA), o **segmento de ponte/simulação** (RP2040) e o **segmento terreno** (dashboard fullstack). A avaliação incide sobre arquitetura, protocolos de comunicação, robustez perante o ambiente de radiação e os compromissos de engenharia assumidos em cada escolha.

1 Visão geral do sistema

O projeto implementa a cadeia completa de um nanossatélite (classe CubeSat), desde o computador de bordo em voo até à consola de operação em terra. É um sistema **ciber-físico distribuído** composto por três segmentos que comunicam por ligações série heterogéneas.

Segmento	Repositório	Papel no sistema
Espacial (OBC)	PS_OBC.X	Firmware <i>bare-metal</i> do computador de bordo. Lê todos os sensores, gere os modos de missão, controla energia e atitude, guarda telemetria em memória externa, envia telemetria por rádio e aceita atualizações de firmware em voo (OTA). Inclui um bootloader dedicado separado.
Ponte / Simulação	solder_test	Firmware para RP2040 (Raspberry Pi Pico). Simula 5 sensores com dinâmica orbital LEO realista e responde-lhes por I ² C multi-endereço (via PIO). Serve ainda de ponte transparente (passthrough) entre a estação terrena (USB/Bluetooth) e o OBC (UART TT&C).
Terreno (Ground Station)	dashboard	Aplicação fullstack de controlo de missão: backend Kotlin/Spring Boot (REST + WebSocket + série), frontend React/TypeScript, base de dados PostgreSQL. Recebe telemetria em tempo real, mostra órbita, estado dos subsistemas e logs, e envia firmware por OTA.

Cadeia de comunicação ponta-a-ponta

```
# Downlink de telemetria e Uplink de comandos/OTA
Sensores/Atuadores      OBC (Cortex-M7)      Ponte RP2040      Estacao Terrena
+-----+ I2C      +-----+ UART      +-----+ USB/BT      +-----+
| GNSS IMU EPS |<----->| Firmware  |<----->| Passthrough|<----->| Kotlin backend|
| Press Temp   | SPI    | bare-metal | 9600    | + sim I2C  | serial  | + React UI   |
| Propulsor    |<----->| FSM/super- | TT&C    | +-----+      | + PostgreSQL |
| W25Q128 QSPI |<----->| loop      |         |         |         | +-----+
+-----+      +-----+      WebSocket --> browser (tempo real)
```

No fluxo típico, o OBC amostra os sensores, compõe uma trama de telemetria de 75 bytes e envia-a pela UART de TT&C. A ponte RP2040 reencaminha os bytes para o dashboard, que os descodifica, persiste em base de dados e difunde para o browser por WebSocket. No sentido inverso, o operador envia comandos (4 bytes) ou uma imagem de firmware que percorre o mesmo caminho até ao bootloader do OBC.

Filosofia de conceção. O firmware assume três princípios que atravessam todo o código: **(i)** execução não-bloqueante (nenhuma operação de I/O pode travar o ciclo principal), **(ii)** memória totalmente estática (zero malloc /heap) e **(iii)** tolerância a falhas de radiação por redundância modular tripla (TMR) em software.

2 Tecnologias e linguagens utilizadas

Segmento	Linguagem / Runtime	Frameworks, bibliotecas e ferramentas
OBC (firmware)	C (C99, bare-metal)	Toolchain XC32 (GCC ARM), MPLAB X IDE, acesso direto a registos (sem Harmony em runtime). CMake/Makefile auxiliares. ~10.600 linhas de C.
Bootloader OTA	C + Assembly de arranque	Projeto MPLAB X separado, <i>linker scripts</i> próprios (<code>samv71q21_boot.ld</code> / <code>_app.ld</code>), driver EFC (Embedded Flash Controller) e QSPI dedicados.
Ponte / Simulação	C (Pico SDK)	Raspberry Pi Pico SDK , PIO (Programmable I/O) para escravo I ² C multi-endereço, <code>multicore</code> , <code>hardware/uart</code> . CMake.
Backend terreno	Kotlin 2.0.21 / JVM (Java 21)	Spring Boot 3.4.5 (Web, WebSocket, Actuator, Data JPA, Security), jSerialComm 2.11.0 (porta série), JJWT 0.12.6 (JWT), Jackson, Gradle (wrapper).
Frontend terreno	TypeScript	React + Vite , WebSocket cliente (hook <code>useWebSocket</code>), componentes de telemetria, órbita, heatmap de sensores, OTA, auditoria e administração.
Persistência	SQL	PostgreSQL 16 (alpine), esquema via <code>init.sql</code> , Hibernate/JPA.
Infraestrutura	YAML / Docker	Docker Compose (postgres + backend Spring + frontend Nginx), <i>healthchecks</i> , perfis (<code>dev</code> / <code>full</code>). Git em cada repositório.

Poliglota por camada. A escolha de linguagem segue a natureza de cada camada: C para o metal (determinismo e controlo de registos), Kotlin/JVM para o backend (produtividade, concorrência e ecossistema Spring) e TypeScript para a UI (segurança de tipos no browser). A fronteira entre elas é sempre um **protocolo série binário/ASCII** bem definido, o que mantém as camadas desacopladas.

3 Arquitetura de software

3.1 Firmware do OBC — modelo de execução

O firmware é **bare-metal**, **sem RTOS**. O `main()` inicializa o clock (300 MHz), corre testes de arranque e entra num **super-loop cooperativo** que chama `mission_lifecycle()` indefinidamente. Cada iteração faz avançar as máquinas de estados de todos os subsistemas por «um passo» (*tick*), sem nunca bloquear.

```
int main(void) {
    hal_clock_init_300mhz();    // PLL -> 300 MHz
    debug_uart_init();
    if (!system_init()) hal_system_reset();
    test_qspi_rw(); test_wcet(); test_seu_injection(); // validacao em placa
    ttc_read_async();           // arranca RX assincrono do radio
    while (1) { mission_lifecycle(); } // super-loop cooperativo
}
```

Organização em camadas

Camada	Pasta	Responsabilidade
Aplicação	src/app/	Lógica de missão, FSM de modos (<code>modeSelector.c</code> , <code>modes.c</code>), agregação de sensores (<code>sensors.c</code>), proteção SEU/TMR (<code>seu.c</code> , <code>seu_data.c</code>).
Periféricos	src/peripherals/	Módulos de alto nível por dispositivo (GNSS, IMU, EPS, pressão, temperatura, propulsor, memória externa, TT&C). Fazem <i>parsing</i> e preenchem estruturas globais; não tocam no hardware.
Drivers	src/drivers/	FSM não-bloqueantes por barramento: <code>i2c_driver</code> , <code>spi_driver</code> , <code>usart_driver</code> , <code>qspi_driver</code> . Gerem <i>timeouts</i> , <i>retries</i> e <i>callbacks</i> .
HAL	src/hal/	Acesso direto a registos do MCU (clocks, GPIO, SysTick, TWIHS, SPI, QSPI, USART). Isola o silício dos drivers.
Config	inc/config/board.h	Endereços I ² C, tamanhos de buffer, parâmetros OTA/QSPI, <code>USE_REAL_HW</code> (real vs. simulado).

A separação **periférico** → **driver** → **HAL** é rigorosa: por exemplo, `eps.c` só conhece o `i2c_driver` , e o driver só conhece o `hal_i2c` . Isto permite trocar de MCU alterando apenas a HAL, e testar a lógica com *mocks* (`USE_REAL_HW 0`).

3.2 Máquina de estados de missão

A missão é modelada como uma FSM de 6 modos. As transições são decididas de forma centralizada em `getMode()` , com base na saúde do sistema (`isSystemSafe()`) e no nível de bateria.

```
nominal --(!safe)--> safe --(bateria<15%)--> ultra_low_power --(timeout)--> decommissioning
| ^                                     |                                     |
|(janela comms)                       |(OTA && bat>70%)                     |(recupera)
```

v | v v
communication <-----> ota_mode safe

Modo	Comportamento	Condição de saída principal
<code>nominal_mode</code>	Amostra sensores, envia telemetria a cada 5 s	Sistema inseguro → <code>safe</code> ; janela aberta → <code>communication</code>
<code>communication_mode</code>	Ativo durante janela de contacto com terra	OTA pedido e bateria >70% → <code>ota</code>
<code>ota_mode</code>	Sub-FSM de 11 estados grava firmware na flash externa	Fim/erro → <code>communication</code>
<code>safe_mode</code>	Reduz atividade, mantém telemetria mínima	Recupera saúde → <code>nominal</code>
<code>ultra_low_power_mode</code>	Bateria crítica (<15%), corta consumos	Timeout de missão → <code>decommissioning</code>
<code>decommissioning_mode</code>	Estado terminal (fim de vida)	—

Limiares de segurança (de `mission.h`): tensão 3,1–6,5 V · corrente 1,0–2,0 A · temperatura –10...60 °C · bateria segura ≥30% · crítica <15% · OTA só com bateria >70%.

3.3 Backend terreno — arquitetura em serviços

O backend Spring segue uma arquitetura em camadas **Controller** → **Service** → **Repository** com forte uso de padrões. O transporte série é deliberadamente «fino»: o `ComPortService` lê bytes crus, coloca-os numa fila (`LinkedBlockingQueue`) e um segundo *thread* alimenta o **codec** selecionado para a ligação.

- **Strategy nos codecs:** `RawBinaryTelemetryCodec` , `AsciiTelemetryCodec` e `AutoTelemetryCodec` implementam a mesma interface `TelemetryCodec` ; uma *factory* escolhe o codec (RAW/ASCII/AUTO) por ligação.
- **Transporte abstraido:** `ComPortService` e `BluetoothService` partilham a mesma interface, pelo que trocar de meio físico não afeta as camadas superiores.
- **Segurança:** `SecurityConfig` + `JwtAuthFilter` (JWT via JJWT), com RBAC (`Role` / `Permission`) e auditoria (`AuditService`).
- **Tempo real:** `SatelliteWebSocketHandler` difunde telemetria/estado/logs/progresso OTA para o browser; persistência assíncrona em PostgreSQL via `TelemetryPersistenceService` .
- **Throttling:** a telemetria é limitada a 1 emissão/s para não inundar o WebSocket e a base de dados, mantendo sempre o valor mais fresco em memória.

4 Interfaces de comunicação (I²C, SPI, UART, QSPI)

O OBC utiliza quatro barramentos distintos, todos implementados como **máquinas de estados não-bloqueantes** com *timeout* e callback. Cada driver expõe uma API `*_async()` + `*_tick()`, chamada uma vez por iteração do super-loop.

Interface	Periférico(s)	Parâmetros	Notas de implementação
I²C (TWIHS0)	GNSS, IMU, EPS, Pressão, Temperatura	25 kHz, PA03=SDA, PA04=SCL	Hardware real via periférico TWIHS0 do SAMV71. FSM com <i>repeated start</i> para leitura por registo e bus recovery automático após 3 erros consecutivos.
SPI	Propulsor (thruster)	Full-duplex, CS=pino 25, divisor 255	FSM com controlo explícito de <i>chip-select</i> , transferência byte-a-byte e verificação de checksum XOR na trama de 9 bytes.
UART/ USART	TT&C (rádio) e Debug	TT&C 9600 8N1 · Debug 115200 8N1	FSM de TX e RX separadas; RX byte-a-byte por interrupção (RXRDY) com buffer circular. Suporta comandos de 4 B, trama de telemetria de 75 B e <i>frame scanner</i> OTA de 134 B.
QSPI	Flash externa W25Q128 (16 MB)	Alta velocidade, retry×3, timeout 5000	FSM com sequência Write-Enable → Check-WEL → comando → Wait-Busy. Escrita por página (≤256 B), <i>erase</i> por setor (4 KB).

4.1 I²C — TWIHS0 e recuperação de barramento

O `hal_i2c` configura o periférico TWIHS0 (pinos PA03/PA04 em função periférica A) e calcula os divisores de clock a partir de MCK=150 MHz. O `i2c_driver` implementa a FSM (`STARTING` → `SELECT_MODE` → `RESTART` → `READ/WRITE` → `STOP`). O tratamento de erro é notável:

```
// Se falhar 3 vezes seguidas, o bus pode estar preso (escravo a segurar SDA)
if (h->error_count >= 3) {
    hal_i2c_bus_recovery(); // gera pulsos de clock para libertar o escravo
    h->error_count = 0;
}
```

4.2 SPI — transferência full-duplex

A FSM SPI (`IDLE` → `CS_LOW` → `TRANSFER` → `WAIT_TX` → `WAIT_RX` → `CS_HIGH` → `STOP`) transfere e recebe em simultâneo (full-duplex). Envia bytes *dummy* (0x00) para «puxar» dados do escravo e valida cada transferência com *timeout*, chamando o callback com 0 (sucesso) ou -1 (erro).

4.3 UART/TT&C — três protocolos sobre a mesma linha

A mesma UART transporta três formatos consoante o modo: **(a)** comandos de 4 bytes (uplink), **(b)** trama de telemetria de 75 bytes (downlink) e **(c)** pacotes OTA de 134 bytes. Um *handshake* de sincronização (`0xAA` `0x55` repetido) e um *frame scanner* dedicado para o OTA permitem multiplexar tudo num único canal a 9600 baud.

4.4 QSPI — flash NOR externa

O driver QSPI abstrai os comandos da W25Q128 (Write-Enable, leitura do *status register*, *page program*, *sector erase*) numa FSM que verifica o **Write-Enable-Latch** antes de escrever e espera o fim de operação (bit BUSY) por *polling* não-bloqueante, com até 3 tentativas.

5 Hardware e respectivas especificações técnicas

5.1 Computador de bordo (OBC)

Componente	Especificação
MCU principal	Microchip ATSAMV71Q21B — ARM Cortex-M7 a 300 MHz , FPU de dupla precisão (FPv5-D16), 2 MB de flash interna, 384 KB SRAM, I-Cache/D-Cache. MCK (clock de periféricos) = 150 MHz.
Flash interna	2 MB. Mapa: bootloader em 0x00400000–0x0040FFFF (64 KB); aplicação em 0x00410000–0x005FFFFF (~1984 KB).
Flash externa	Winbond W25Q128 — 16 MB NOR SPI/QSPI. Regiões: config (64 KB), logs de telemetria (~960 KB), metadados OTA (0x100000) e imagem OTA (0x101000).
Programação/Debug	JTAG/SWD via ATMEL-ICE ou MPLAB PICKIT 4.

5.2 Periféricos de missão (payload e plataforma)

Periférico	Barramento	Endereço/ CS	Grandezas medidas / função
GNSS (GPS)	I ² C	0x42	Latitude, longitude, altitude, velocidade (posição orbital)
IMU	I ² C	0x68	Aceleração (ax/ay/az), giroscópio (gx/gy/gz), magnetómetro (mx/my/mz) — atitude
Sensor de pressão	I ² C	0x77	Pressão (hPa/bar)
Sensor de temperatura	I ² C	0x48	Temperatura (°C)
EPS (energia)	I ² C	0x60	Tensão e corrente do barramento de potência
Propulsor (cold-gas)	SPI	CS pino 25	Estado, pressão, temperatura, impulso (N), estado da válvula
FlexiForce	ADC	—	Força/pressão analógica (sensor de força)
Memória externa	QSPI	—	Registo persistente de telemetria e <i>staging</i> de firmware OTA
TT&C (rádio)	UART	—	Telecomando, telemetria e rastreio (downlink/uplink)

5.3 Ponte RP2040 (banco de ensaio)

O **Raspberry Pi Pico (RP2040, dual-core Cortex-M0+)** substitui, em bancada, todo o hardware de sensores. Usa os blocos **PIO** para implementar um escravo I²C capaz de responder aos 5 endereços em simultâneo, e simula dinâmica orbital LEO realista (órbita ~550 km, inclinação 51,6°, período ~96 min, ciclo sol/eclipse -20...+45 °C, tensão/corrente do EPS a seguir a iluminação solar). Reencaminha ainda a UART de TT&C entre a estação terrena (USB ou módulo Bluetooth HC-05) e o OBC.

5.4 Infraestrutura da estação terrena

Servidor JVM (Java 21) a correr o backend Spring na porta 8080, PostgreSQL 16 na 5432 e frontend Nginx na porta 80/5173. A ligação física ao satélite (ou à ponte) faz-se por porta série COM através da biblioteca jSerialComm. Todo o conjunto pode ser levantado com Docker Compose.

6 Fluxo de dados e integração entre componentes

6.1 Aquisição e agregação a bordo

Em cada *tick*, `sensors_tick()` orquestra a leitura de todos os subsistemas e envolve o ciclo em proteção TMR:

```
void sensors_tick(void) {
    sensors_seu_register();      // regista structs globais sob TMR (1a vez)
    seu_data_scrub();           // votacao 2-de-3 + reparacao de SEUs
    eps_tick(); gnss_tick(); imu_tick(); pressure_tick();
    temperature_tick(); ttc_tick(); propulsor_tick(); ExtMem_Tick();
    seu_data_commit();          // consolida escritas legitimas nas 3 copias
}
```

Os periféricos I²C são lidos em bloco; o SPI (propulsor) alterna com os I²C via `spi_turn` para não saturar um único *tick*. Cada callback de conclusão faz o *parsing* dos bytes crus e escreve na respetiva estrutura global (`eps` , `imu` , ...), que é a «fonte de verdade» partilhada.

6.2 Telemetria: da RAM ao operador

1. A cada 5 s (modo nominal), `ttc_send_telemetry()` serializa 18 *floats* em **big-endian** numa trama de 75 bytes com *checksum* XOR e envia-a pela UART.
2. Em paralelo, a telemetria é guardada na flash externa (`ExtMem_SaveTelemetryAsync`) num buffer de logs, respeitando o limite de 256 B por página.
3. A ponte RP2040 reencaminha os bytes para o dashboard.
4. O `ComPortService` descodifica (codec RAW/ASCII/AUTO), aplica *throttle* de 1/s, persiste em PostgreSQL e difunde por WebSocket.
5. O frontend React atualiza painéis de telemetria, órbita, heatmap e logs em tempo real.

6.3 Atualização OTA — integração de ponta a ponta

O OTA é o fluxo mais complexo por atravessar todos os segmentos e o bootloader. O `OtaService` (Kotlin) constrói pacotes de 134 bytes; o `otaMode()` (sub-FSM de 11 estados no OBC) grava-os na flash externa; o bootloader aplica-os no arranque seguinte.

```
Dashboard --> Pico (passthrough) --> OBC otaMode() --> reboot --> Bootloader
1. CMD_START_OTA (0x10)                4. grava firmware em 0x101000 (flash ext.)
2. N x pacote [AA55][seq][len][128B] 5. verifica CRC32 na flash externa
3. pacote END [seq=FFFF][size|crc|ver] 6. grava metadata (magic=0xAB12CD34) por ULTIMO
                                     7. hal_system_reset()
Bootloader: le metadata --> valida CRC --> copia p/ flash interna (EFC) --> salta p/ app
```

O protocolo é **stop-and-wait com ACK/NACK**: o OBC responde `0xAC` (ACK) por pacote válido ou `0x4E` (NACK) para pedir retransmissão; o backend tenta até 3 vezes por pacote, com *timeout* de 3 s. A metadata

só é escrita **depois** de o firmware estar completo e com CRC verificado — se algo falhar a meio, o satélite arranca a versão anterior (rollback implícito).

Integração desacoplada. Backend e firmware partilham apenas constantes de protocolo (`CMD_START_OTA=0x10` , códigos ACK/NACK, tamanho de pacote 134 B). Não há dependência de código entre segmentos — a fronteira é o formato binário na linha série, o que permite evoluir cada lado de forma independente (por exemplo, trocar a ponte USB por antenas reais sem mexer no OBC).

7 Forma de comunicação de cada periférico

Cada periférico define o seu próprio «micro-protocolo» ao nível dos bytes: endereço/registo, ordem e escala dos dados, e verificação de integridade. Segue-se o detalhe de cada um.

7.1 EPS — Electrical Power System (I²C)

Leitura por registo (`use_reg=1`): endereço 0x60, registo 0x09, 2 bytes. Conversão para grandezas físicas por escala fixa:

```
eps.voltage = buf[0] / 10.0f;    // 0xAA (170) -> 17.0 V
eps.current = buf[1] / 100.0f;   // 0x01 (1)   -> 0.01 A
```

7.2 GNSS e IMU (I²C, 18 bytes)

GNSS (0x42) e IMU (0x68) devolvem blocos de 18 bytes decodificados em latitude/longitude/altitude/velocidade e em aceleração/giroscópio/magnetómetro (3 eixos cada), respetivamente.

7.3 Pressão e temperatura (I²C, 2 bytes)

Sensores simples de 2 bytes: pressão em 0x77 e temperatura em 0x48, convertidos para hPa e °C por escala.

7.4 Propulsor (SPI, trama de 9 bytes com checksum)

O OBC envia um byte de comando (`0x01` = pedir telemetria; `0x02` = abrir válvula) seguido de *dummies*, e lê uma trama de 9 bytes, validada por XOR:

```
// Frame (9 bytes):
[0]  STATUS    0x00=IDLE 0x01=FIRING 0x02=ERROR
[1-2] PRESSURE raw/100 -> bar          [5-6] THRUST raw/10 -> N
[3-4] TEMP     raw/10  -> C          [7]  VALVE  0=fechada 1=aberta
[8]  CHECKSUM = XOR de [0..7]
```

7.5 Memória externa W25Q128 (QSPI)

Comunicação por comandos SPI-flash normalizados: `WREN` (Write-Enable), leitura do *status register* (para confirmar WEL e bit BUSY), *page program* (≤256 B), *sector erase* (4 KB) e leitura. O módulo `ext_memory.c` mantém um ponteiro de escrita circular na região de logs e limpa a *write-protection* (WRSR) antes de operações OTA.

7.6 TT&C — três tramas sobre UART

Trama	Tamanho	Estrutura
Comando (uplink)	4 bytes	Código (ex.: <code>0x10</code> START_OTA, <code>0x11</code> END_OTA, <code>0x20</code> REQUEST_DATA) + argumentos

Telemetria (downlink)	75 bytes	[0x20] + 18 floats big-endian (V, I, lat, lon, alt, spd, a×3, g×3, m×3, press, temp, doppler) + estado + [X0R]
Pacote OTA	134 bytes	[0xAA][0x55] + seq (2B BE) + len (2B BE) + payload 128 B
ACK / NACK / READY	4 bytes	0xAC (ok) / 0x4E (retransmitir) / 0xE0 (flash apagada, pronto)

7.7 Simulação I²C na ponte (RP2040/PIO)

Do lado do banco de ensaio, o RP2040 usa programas **PIO** para detetar condições de *start/stop* e transacionar bytes, respondendo aos 5 endereços I²C como se fossem 5 escravos físicos independentes — permitindo testar todo o firmware do OBC sem os sensores reais.

8 Vantagens e desvantagens das abordagens

Análise crítica das principais decisões de engenharia, contrastando o benefício teórico com o custo prático observável no código.

8.1 Super-loop cooperativo *bare-metal* (sem RTOS)

Vantagens

- Determinismo total: sem *scheduler*, o tempo de ciclo é previsível e mensurável (há `wcet_test` para o comprovar).
- Sem *overhead* de kernel nem risco de *deadlocks*/ inversão de prioridade.
- Footprint mínimo de RAM/flash — crítico num MCU.

Desvantagens

- Toda a lógica tem de ser fatiada em FSM não-bloqueantes: mais complexa de escrever e ler (a sub-FSM OTA tem 11 estados).
- Uma tarefa «mal comportada» que demore atrasa *todas* as outras — não há preempção.
- Latência de resposta = pior caso da soma de todos os *ticks*.

8.2 Drivers não-bloqueantes com callbacks

Vantagens

- Nenhum I/O trava o super-loop; um sensor avariado apenas expira por *timeout* em vez de congelar o satélite.
- API uniforme (`*_async` + `*_tick` + callback) em todos os barramentos.
- **Bus recovery** automático no I²C e *retries* no QSPI aumentam a robustez em voo.

Desvantagens

- Fluxo de controlo fragmentado por callbacks e estados — depuração mais difícil.
- Estado espalhado por variáveis `static` : exige disciplina para evitar condições de corrida lógicas entre *ticks*.

8.3 Tolerância a radiação por TMR/SEU em software

Vantagens

- Protege dados críticos contra *Single Event Upsets* sem hardware *rad-hard* dispendioso — votação 2-de-3 com reparação em cada *tick*.
- Cópias em bancos de SRAM fisicamente separados reduzem a chance de um evento atingir duas cópias.
- Injeção de falhas testável (`seu_injection_test`).

Desvantagens

- Triplica o uso de RAM e adiciona custo de CPU por *scrub/commit* a cada ciclo.
- Não protege a execução do próprio código nem os registos do CPU (só os dados registados).
- A separação «física» das cópias depende do *layout* do linker, não é garantida por hardware.

8.4 Integridade: XOR na telemetria vs. CRC32 no OTA

Vantagens

- XOR é baríssimo em CPU e suficiente para detectar erros de bit isolados na telemetria de alta cadência.
- CRC32 (poly 0xEDB88320) no OTA garante integridade forte onde ela é imprescindível (código executável).
- Escolha proporcional ao risco: gasta-se cálculo onde importa.

Desvantagens

- XOR não detecta erros de bits pares nem trocas de ordem — telemetria corrompida pode passar despercebida.
- Dois esquemas diferentes aumentam a superfície de manutenção do protocolo.

8.5 OTA com bootloader dedicado + *staging* em flash externa

Vantagens

- Atualização atômica e segura: a metadata (magic) é escrita por último e o CRC é verificado antes de aplicar — falhas resultam em **rollback** para o firmware anterior.
- O bootloader é pequeno e isolado (64 KB), reduzindo o risco de se «entijolar» o satélite.
- Resiliência: falha de escrita mantém a flag para *retry* na reinicialização seguinte.

Desvantagens

- Complexidade elevada: dois projetos MPLAB, *linker scripts* distintos e a app tem de arrancar de 0x00410000 (erro aqui = não arranca).
- Consome ~1 MB da flash externa e exige gestão cuidadosa de setores/páginas.
- Protocolo **stop-and-wait** é simples mas lento a 9600 baud (ver 8.6).

8.6 UART TT&C a 9600 baud e I²C a 25 kHz

Vantagens

- Baixas velocidades toleram muito melhor ruído, atenuação e cabos longos — coerente com um enlace de rádio real e com bancada de solda.
- Reduzem erros de *bit* e simplificam a temporização dos *timeouts*.

Desvantagens

- 9600 baud tornam o OTA moroso (uma imagem de ~200 KB demora minutos com ACK por pacote).
- Os 25 kHz de I²C ficam muito abaixo dos 400 kHz previstos no *roadmap* — limitam a cadência de amostragem se os sensores forem muitos.

8.7 Ponte RP2040 como simulador + passthrough

Vantagens

- Permite validar 100% do firmware do OBC sem os sensores reais nem o rádio, com dinâmica orbital realista — excelente para desenvolvimento e demonstração.
- O *passthrough* transparente desacopla a estação terrena do meio físico (USB, Bluetooth, antenas).

Desvantagens

- Introduz um elo extra que pode mascarar problemas de temporização/ruído do hardware real.
- A fidelidade da simulação limita a validação: comportamentos de sensores reais podem divergir do modelo.

8.8 Backend Kotlin/Spring modular (codecs + transporte abstraído)

Vantagens

- Padrão *Strategy* nos codecs e interface comum de transporte tornam trivial acrescentar formatos ou meios (o próprio README prevê um `AntennaService` sem mexer no frontend).
- WebSocket em *push* (em vez de *polling*) dá tempo real eficiente; JWT+RBAC e auditoria dão segurança de nível profissional.
- *Throttling* e fila de bytes protegem a UI e a base de dados de rajadas.

Desvantagens

- Stack pesado (JVM + Spring + PostgreSQL + Docker) para um dispositivo de operador — mais recursos e arranque mais lento que uma solução leve.
- Concorrência com múltiplos *executors/threads* e fila exige cuidado para evitar *backpressure* e perda de mensagens sob carga.

Síntese. As escolhas formam um conjunto coerente e maduro para um projeto acadêmico com ambição de voo: privilegiam **determinismo e robustez** a bordo (bare-metal, FSM não-bloqueantes, TMR, OTA com rollback) e **produtividade e extensibilidade** em terra (Kotlin/Spring, codecs, WebSocket). Os principais pontos a amadurecer são o débito das ligações série (9600 baud / 25 kHz), a substituição do XOR por CRC na telemetria crítica e a redução da complexidade da cadeia OTA — todos compromissos conscientes, não descuidos.